Universidade Cruzeiro do Sul

TÓPICOS AVANÇADOS EM SISTEMAS DE INFORMAÇÃO II

Prof. José Claudio de Sousa
jcsousa@cruzeirodosul.edu.br

Universidade Cruzeiro do Sul

ARQUITETURA DE DESENVOLVIMENTO DE SISTEMAS

DESIGN PATTERNS

PADRÕES CLÁSSICOS OU PADRÕES GoF

O livro "Design Patterns“(1994) de Erich Gamma, John Vlissides, Ralph Jonhsone Richard Helm, descreve 23 padrões de design[2]

- São soluções genéricas para os problemas mais comuns do desenvolvimento de software orientado a objetos;
- O livro tornou-se um clássico na literatura orientada a objeto e continua atual;
- Não são invenções. São documentação de soluções obtidas através da experiência. Foram coletados de experiências de sucesso na indústria de software, principalmente de projetos em C++ e SmallTalk;
- Os quatro autores, são conhecidos como "The Gang of Four", ou GoF ;

POR QUE APRENDER PADRÕES

Aprender com a experiência dos outros

- Identificar problemas comuns em engenharia de software e utilizar soluções testadas e bem documentadas
- Utilizar soluções que têm um nome: facilita a comunicação, compreensão e documentação
- Aprender a programar bem com orientação a objetos
- Os 23 padrões de projeto "clássicos" utilizam as melhores práticas em OO para atingir os resultados desejados
- Desenvolver software de melhor qualidade
- Os padrões utilizam eficientemente polimorfismo, herança, modularidade, composição, abstração para construir código reutilizável, eficiente, de alta coesão e baixo acoplamento

POR QUE APRENDER PADRÕES

Vocabulário comum

- Faz o sistema ficar menos complexo ao permitir que se fale em um nível mais alto de abstração
- Ajuda na documentação e na aprendizagem
- Conhecendo os padrões de projeto torna mais fácil a compreensão de sistemas existentes
- "As pessoas que estão aprendendo POO freqüentemente reclamam que os sistemas com os quais trabalham usam herança de forma complexa e que é difícil de seguir o fluxo de controle. Geralmente a causa disto é que eles não entendem os padrões do sistema" [GoF]
- Aprender os padrões ajudam um novato a agir mais como um especialista

PADRÕES GoF: FORMAS DE CLASSIFICAÇÃO

Há várias formas de classificar os padrões. Gamma et al[2] os classifica de duas formas

- Por propósito:

(1) Criação de classes e objetos, (2) alteração da estrutura de um programa, (3) controle do seu comportamento

- Por escopo: classe ou objeto

PADRÕES GoF: FORMAS DE CLASSIFICAÇÃO

Metsker[1] os classifica em 5 grupos, por intenção (problema a ser solucionado):

- (1) oferecer uma interface,
- (2) atribuir uma responsabilidade,
- (3) realizar a construção de classes ou objetos
- (4) controlar formas de operação
- (5) implementar uma extensão para a aplicação

Padrões

CLASSIFICAÇÃO DOS 23 PADRÕES SEGUNDO GoF

		Propósito		
		1. Criação	2. Estrutura	3. Comportamento
Escopo	Classe	Factory Method	Class Adapter	Interpreter Template Method
	Objeto	Abstract Factory Builder Prototype Singleton	Object Adapter Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

* Padrões "clássicos" selecionados e organizados por Gamma et al. "Design Patterns" [2]

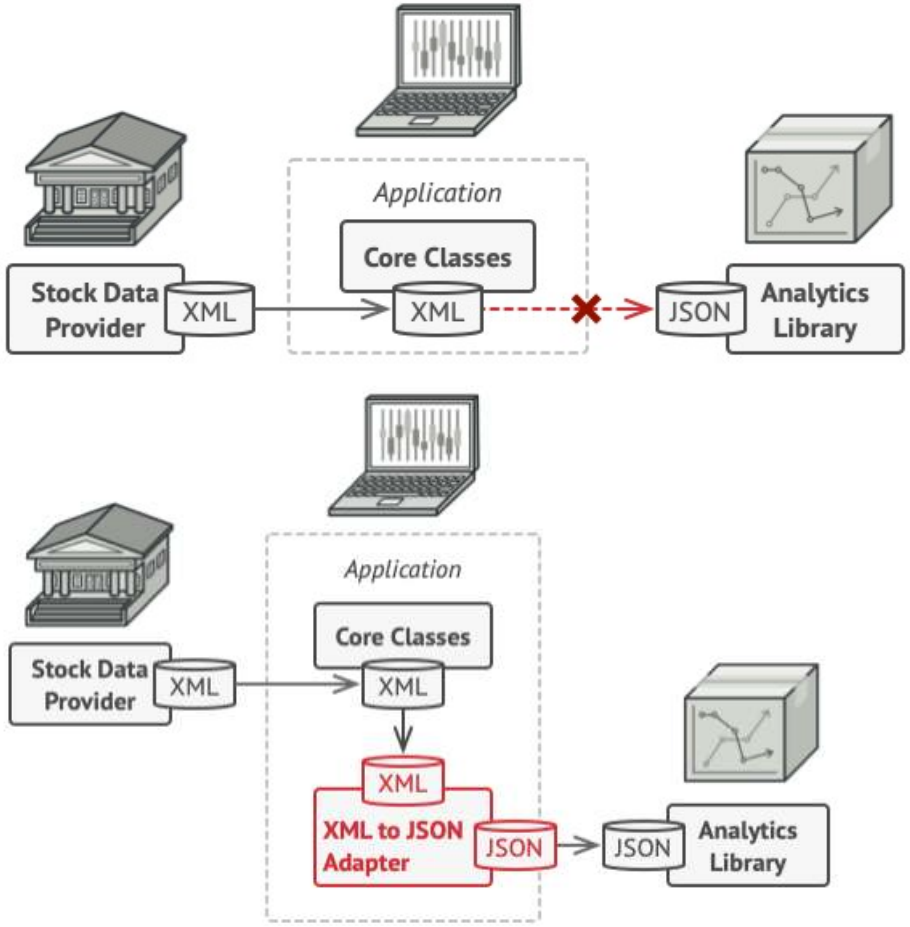
Universidade Cruzeiro do Sul

CLASSIFICAÇÃO DOS 23 PADRÕES SEGUNDO METSKER[I]

Intenção	Padrões
1. Interfaces	Adapter, Facade, Composite, Bridge
2. Responsabilidade	Singleton, Observer, Mediator, Proxy, Chain of Responsibility, Flyweight
3. Construção	Builder, Factory Method, Abstract Factory, Prototype, Memento
4. Operações	Template Method, State, Strategy, Command, Interpreter
5. Extensões	Decorator, Iterator, Visitor

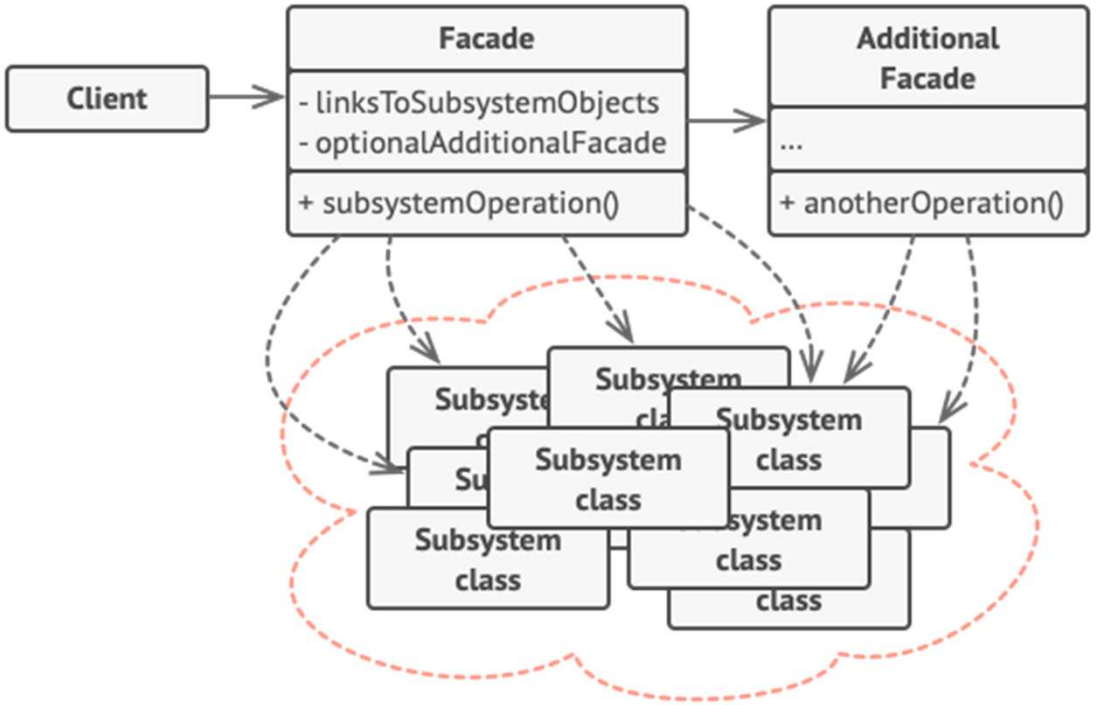
INTERFACES

•**Adapter**: Converte a interface de uma classe em outra que o cliente espera. Use quando você quer que classes com interfaces incompatíveis trabalhem juntas.



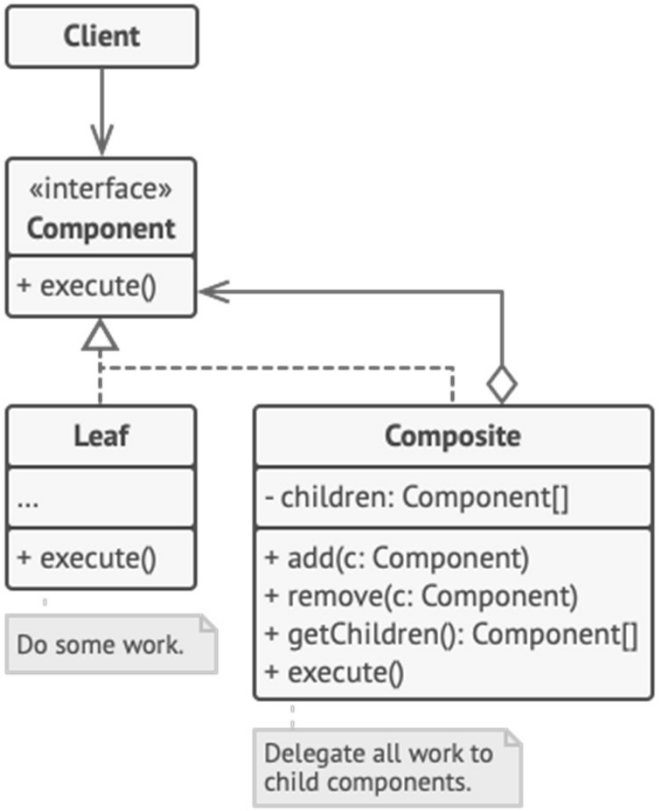
INTERFACES

•**Facade**: Fornece uma interface simplificada para um conjunto complexo de classes ou um subsistema. Use quando quiser reduzir a complexidade de interações com subsistemas.



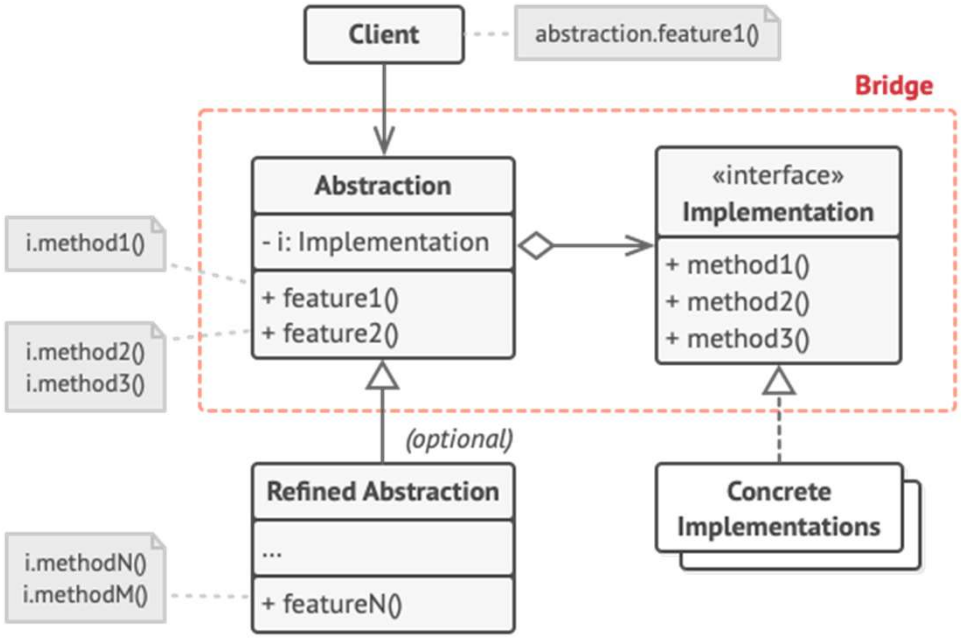
INTERFACES

•**Composite**: Organiza objetos em uma estrutura de árvore para representar hierarquias parte-todo. Use quando você deseja que o cliente trate objetos individuais e composições de objetos da mesma forma.



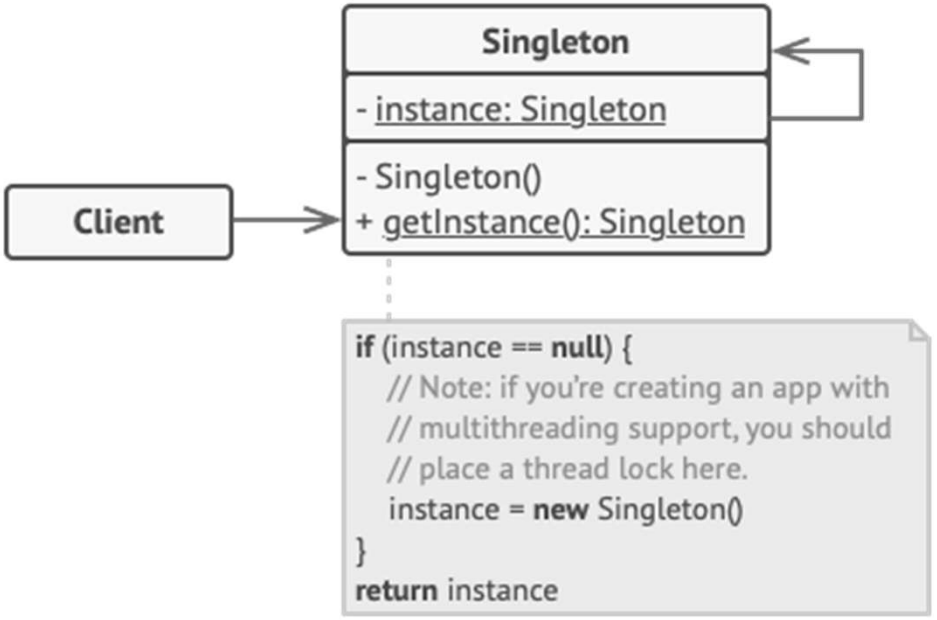
INTERFACES

•**Bridge**: Desacopla uma abstração de sua implementação para que ambos possam variar independentemente. Use quando você precisa separar a abstração da implementação.



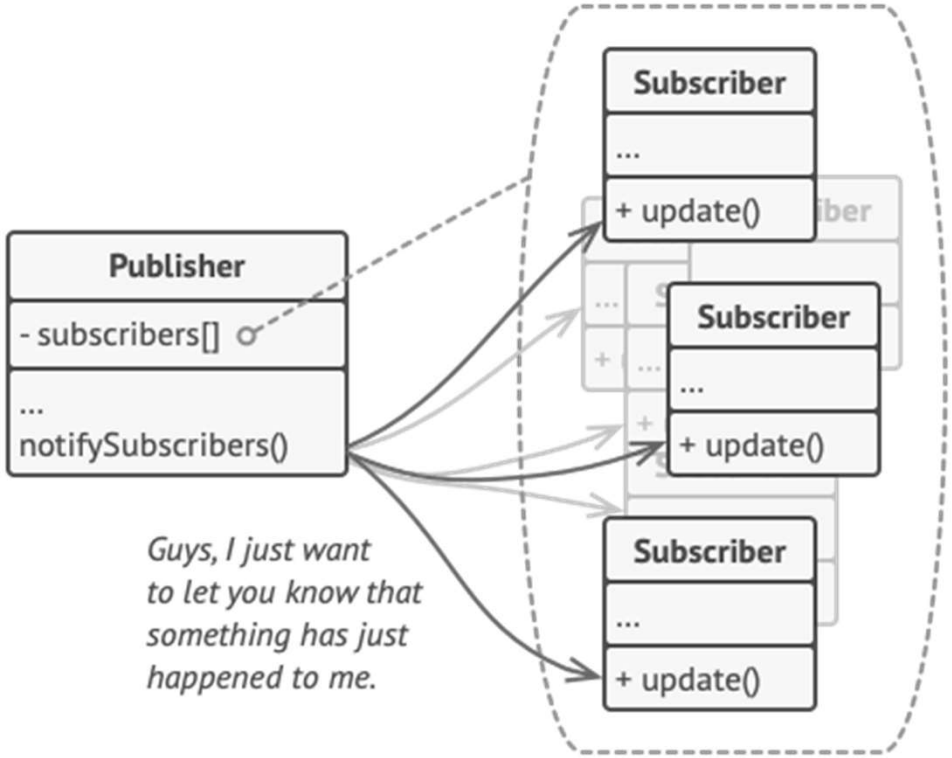
RESPONSABILIDADE

- **Singleton**: Garante que uma classe tenha apenas uma instância e fornece um ponto de acesso global a ela. Use quando precisa de controle sobre uma única instância de uma classe.



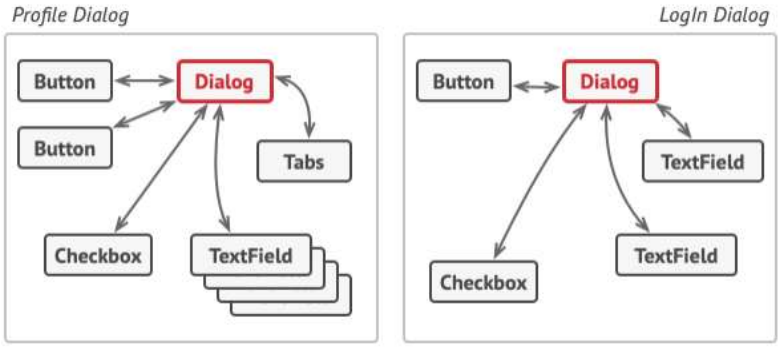
RESPONSABILIDADE

•**Observer**: Define uma dependência um-para-muitos entre objetos para que, quando um objeto muda de estado, todos os seus dependentes sejam notificados automaticamente. Use para cenários como assinaturas de eventos.

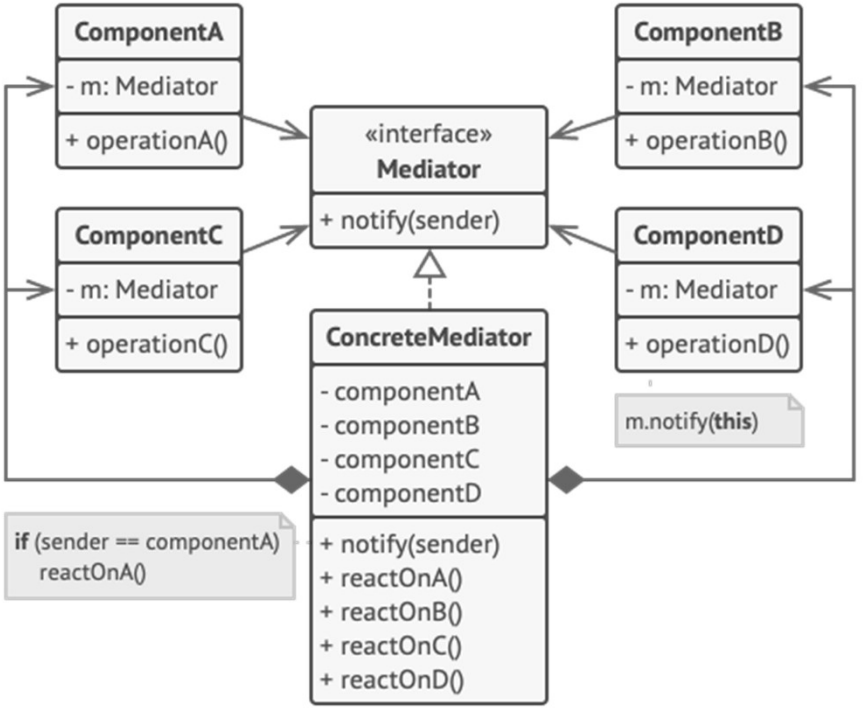


RESPONSABILIDADE

Mediator: Define um objeto que centraliza a comunicação entre outros objetos, evitando a comunicação direta entre eles. Use para reduzir dependências entre objetos.

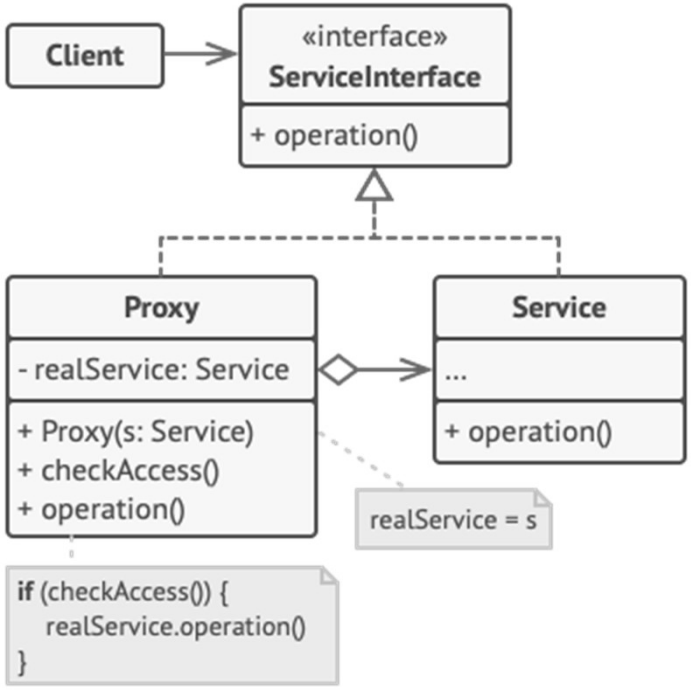


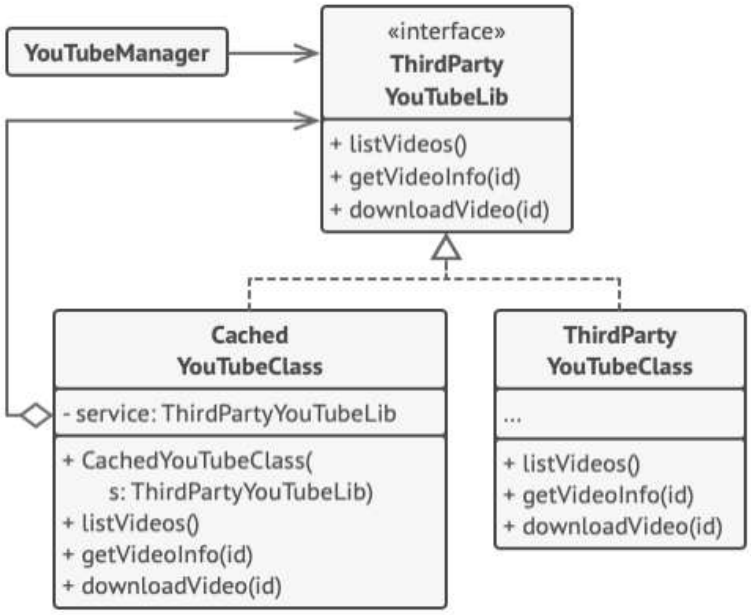
Os elementos da interface do usuário devem se comunicar indiretamente, por meio do objeto mediador.



RESPONSABILIDADE

Proxy: Fornece um substituto para controlar o acesso a outro objeto. Use quando precisar de controle adicional sobre a interação com um objeto, como para lazy loading ou controle de acesso.

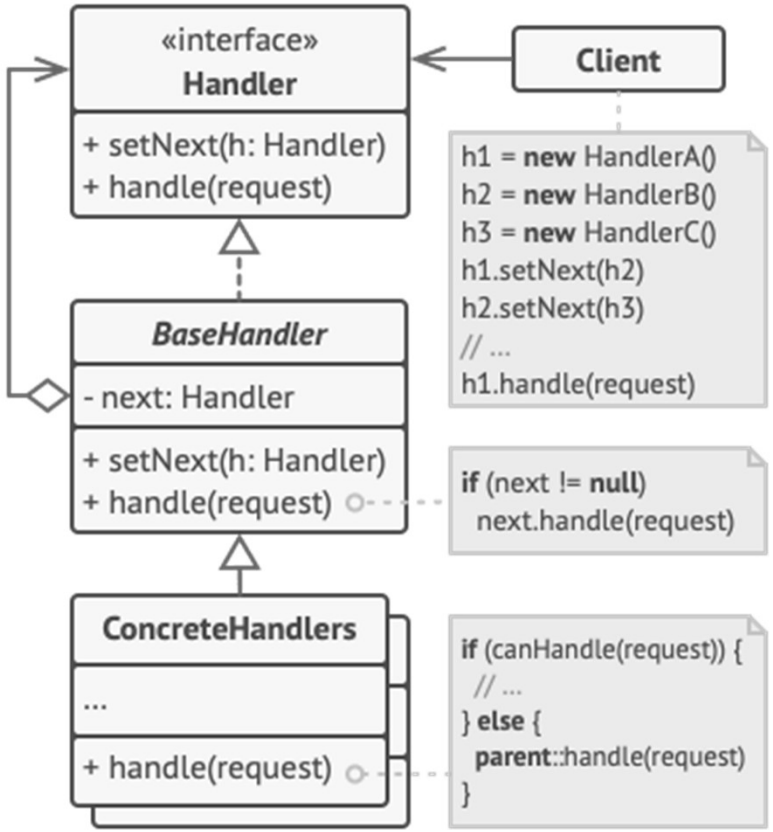




Armazenando em cache os resultados de um serviço com um proxy.

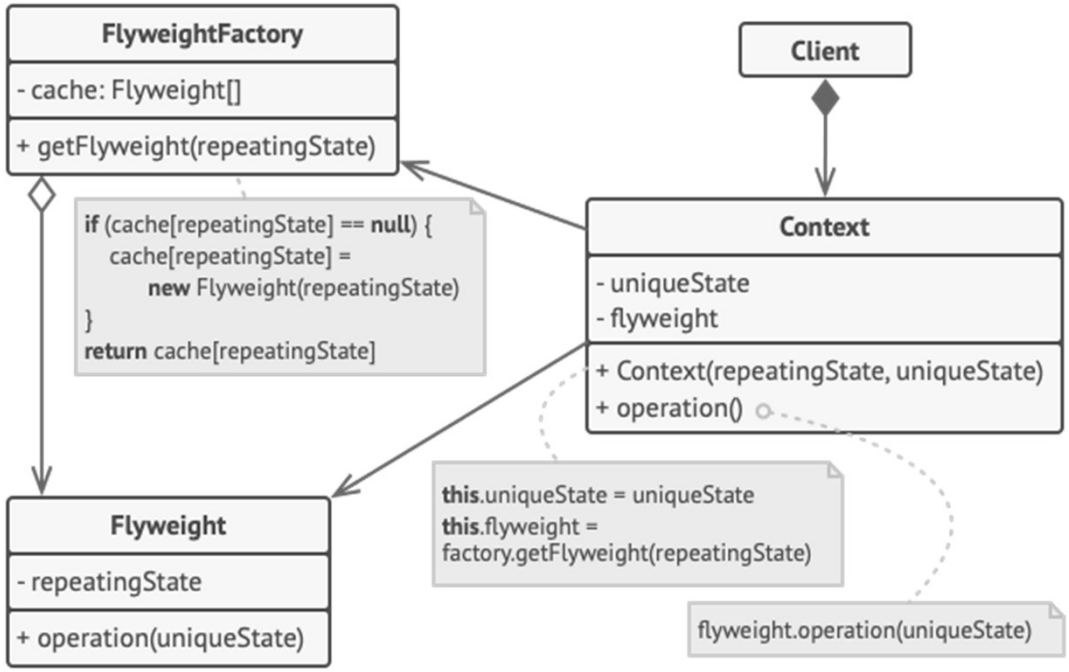
RESPONSABILIDADE

Chain of Responsibility: Passa uma solicitação ao longo de uma cadeia de manipuladores, até que um deles a processe. Use quando você deseja que múltiplos objetos tenham a chance de processar uma solicitação.



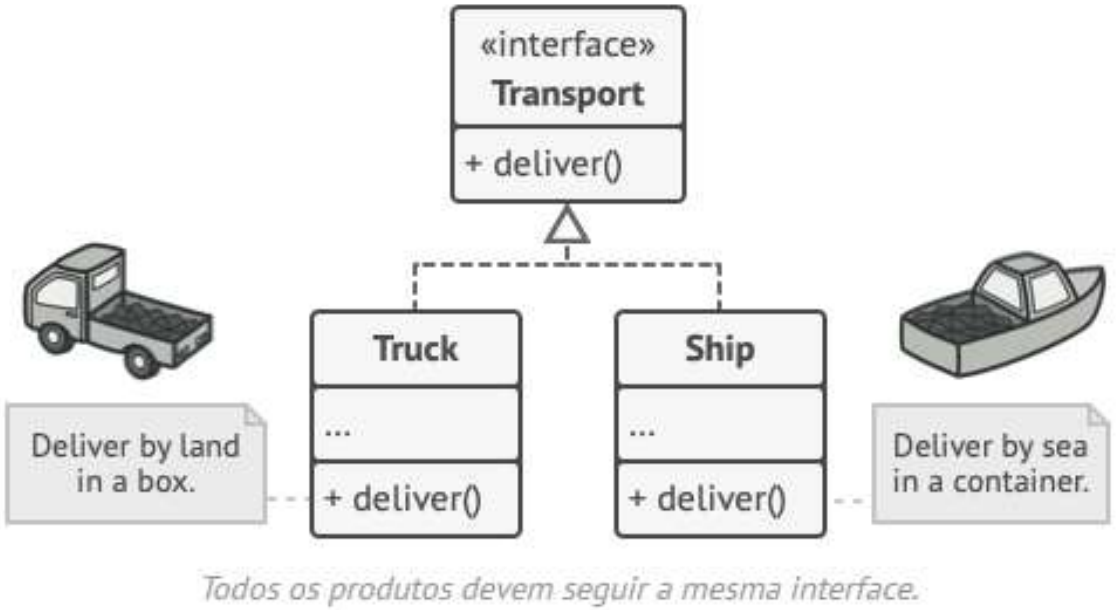
RESPONSABILIDADE

Flyweight: Usa o compartilhamento para suportar eficientemente um grande número de objetos de granularidade fina. Use quando há necessidade de economizar memória.



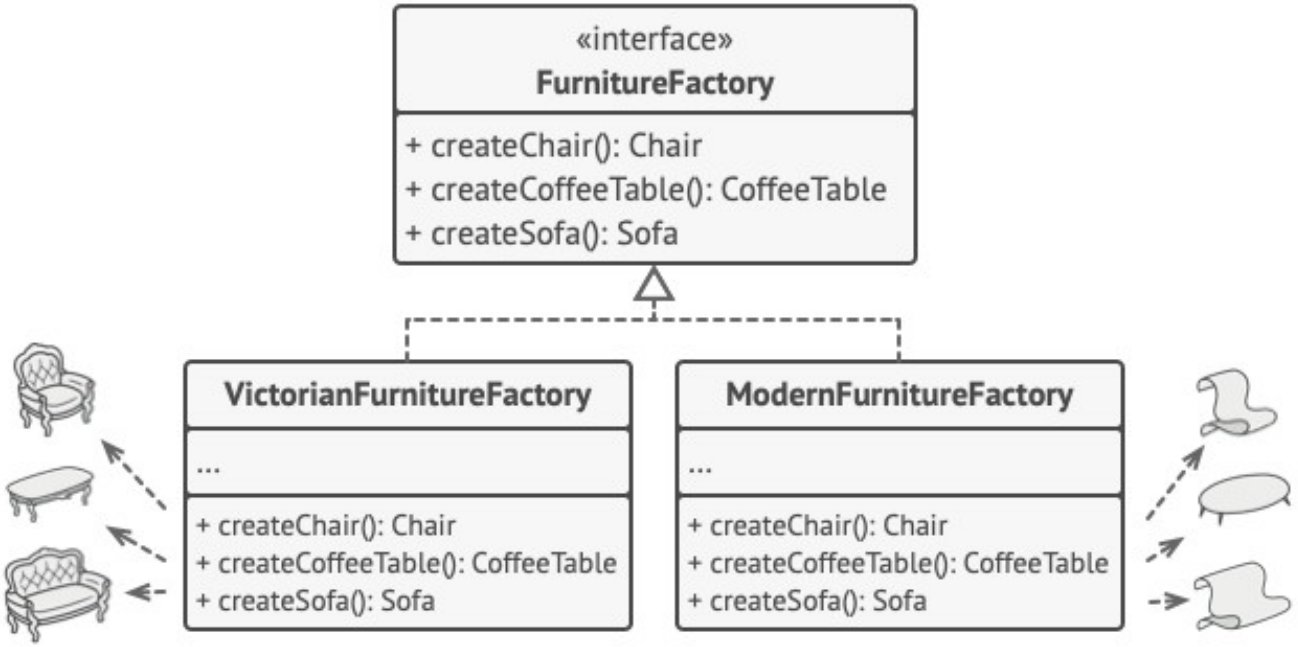
CONSTRUÇÃO

Factory Method: Define uma interface para criar objetos, mas permite que as subclasses alterem o tipo de objetos que serão criados. Use para delegar a lógica de criação de objetos para subclasses.



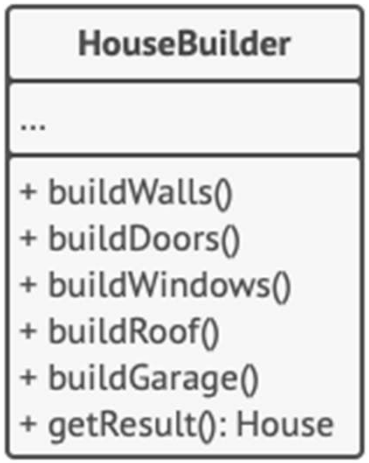
CONSTRUÇÃO

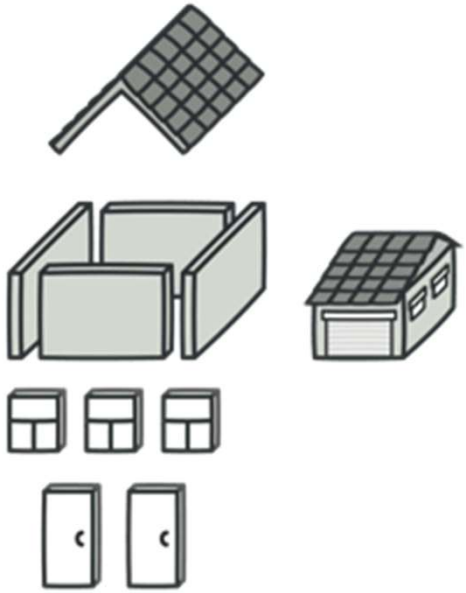
Abstract Factory: Fornece uma interface para criar famílias de objetos relacionados ou dependentes sem especificar suas classes concretas. Use quando um sistema precisa ser independente de como seus objetos são criados.



CONSTRUÇÃO

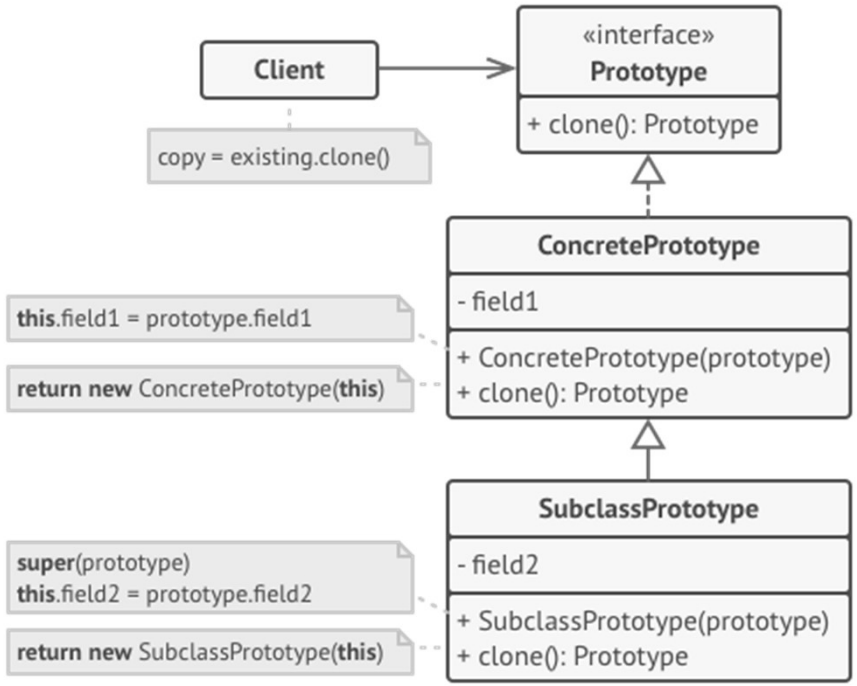
Builder: Separa a construção de um objeto complexo da sua representação, permitindo que o mesmo processo de construção crie diferentes representações. Use para criar objetos complexos passo a passo.





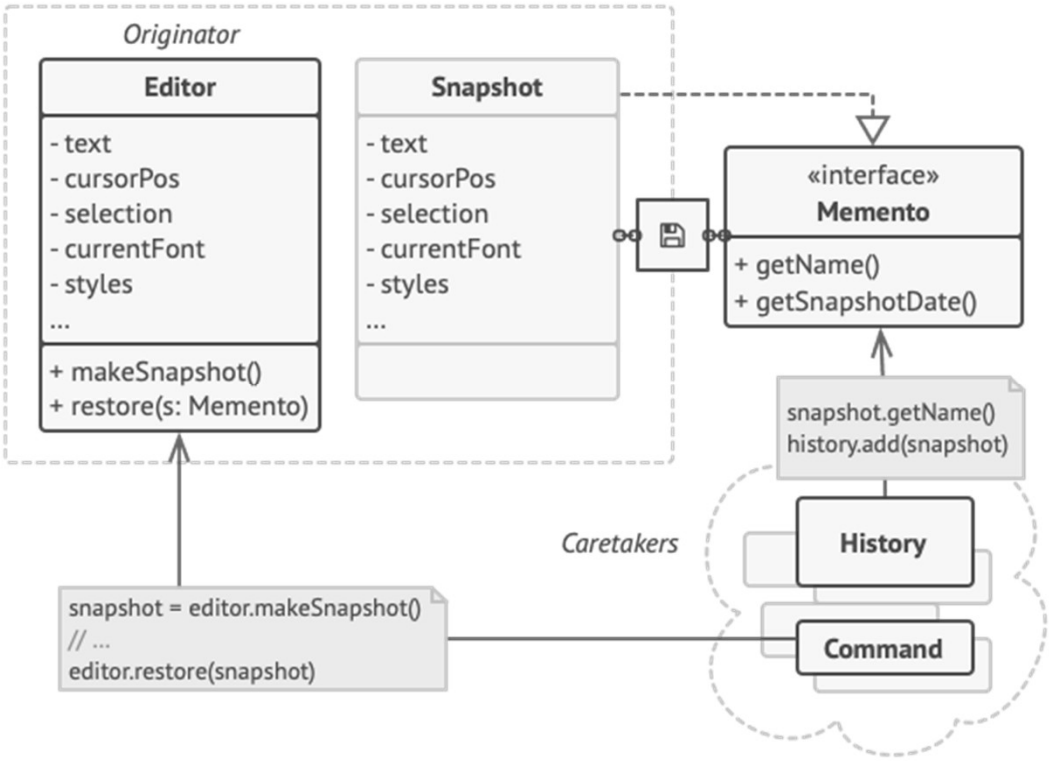
CONSTRUÇÃO

Prototype: Cria novos objetos clonando um protótipo. Use quando a criação de objetos é cara ou complexa e você pode usar um protótipo como base.



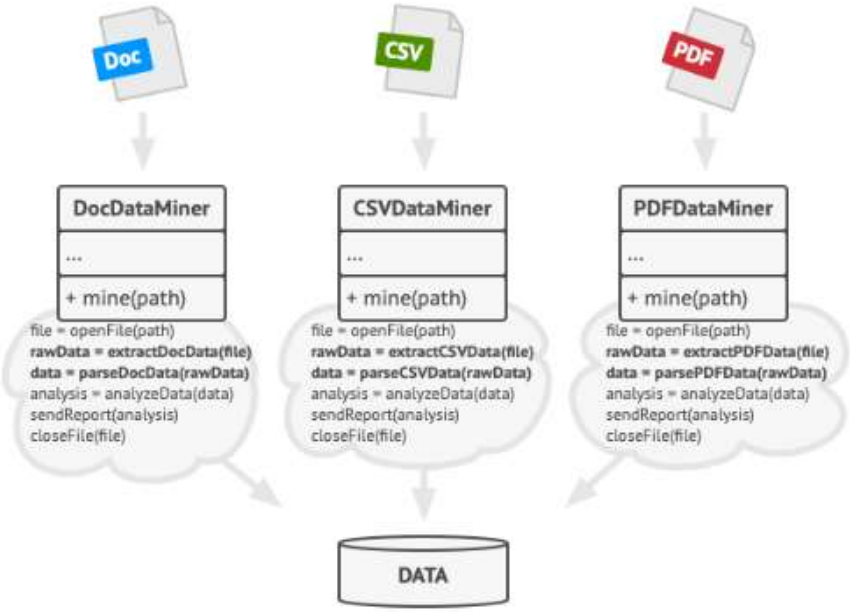
CONSTRUÇÃO

Memento: Captura e restaura o estado interno de um objeto sem violar o encapsulamento. Use para implementar "desfazer" ou "restaurar".



OPERAÇÕES

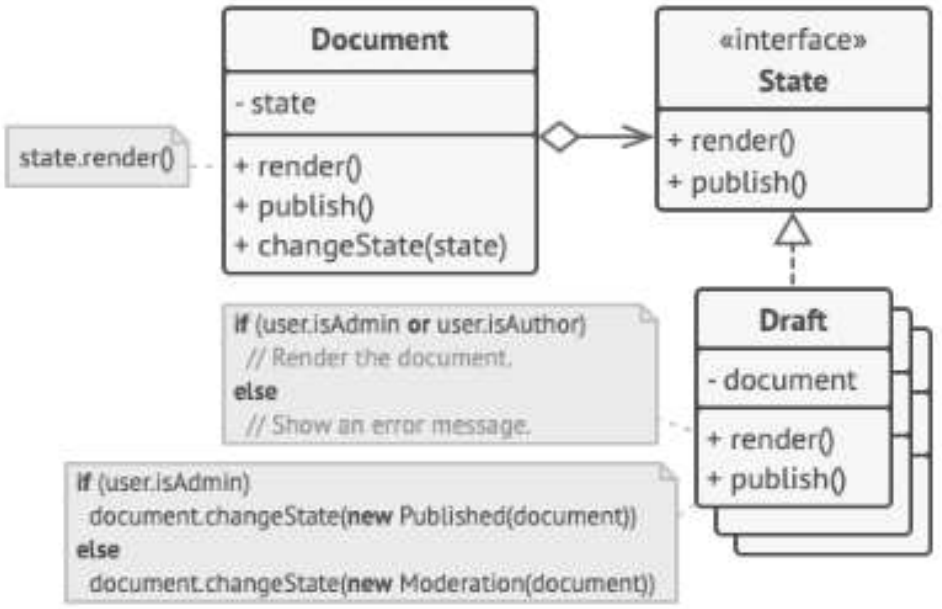
Template Method: Define o esqueleto de um algoritmo, permitindo que subclasses alterem certos passos sem mudar a estrutura geral. Use para reutilizar algoritmos padrão com variações específicas.



Data mining classes contained a lot of duplicate code.

OPERAÇÕES

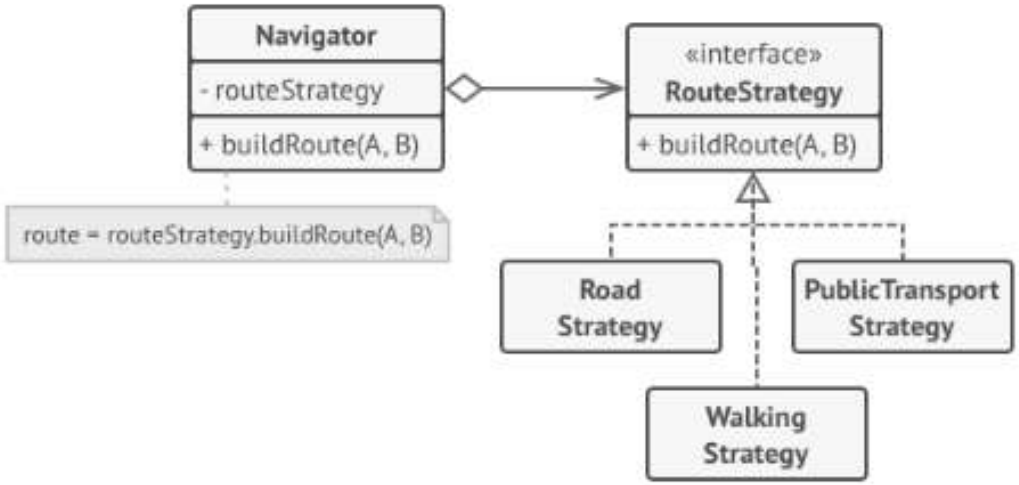
State: Permite que um obje altere seu comportamen quando seu estado interr muda. Use quando comportamento de um obje depende de seu estado.



Document delegates the work to a state object.

OPERAÇÕES

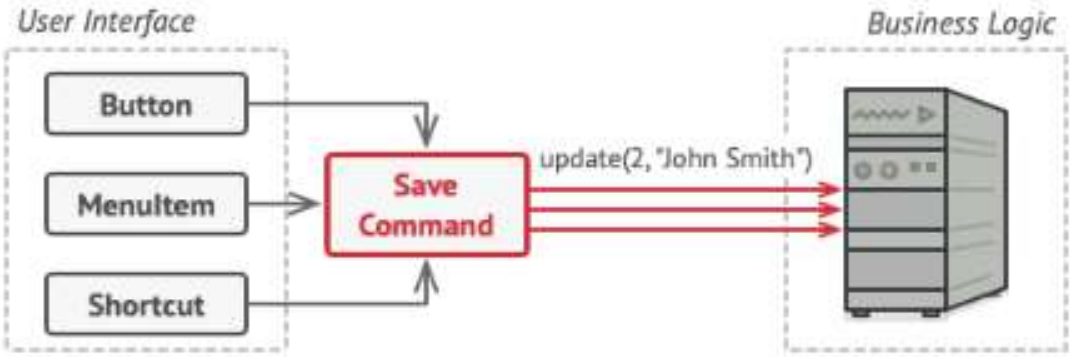
Strategy: Define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis. Use para selecionar dinamicamente um algoritmo ou comportamento em tempo de execução.



Route planning strategies.

OPERAÇÕES

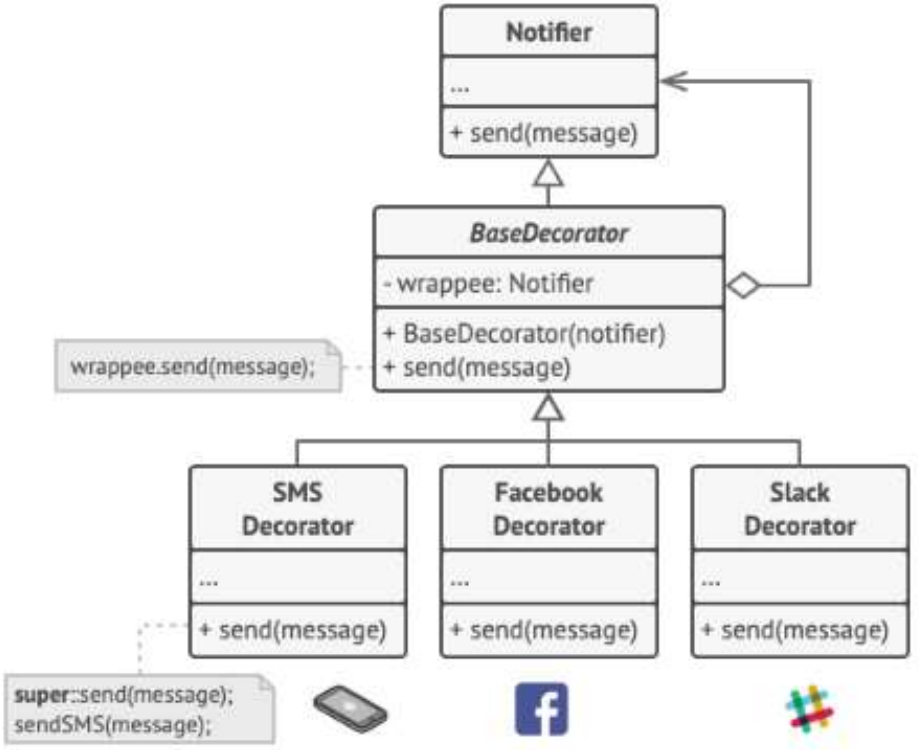
Command: Encapsula uma solicitação como um objeto, permitindo parametrizar clientes com diferentes solicitações, enfileirar ou registrar operações. Use para implementar operações "desfazer" e filas de comandos.



Accessing the business logic layer via a command.

EXTENSÕES

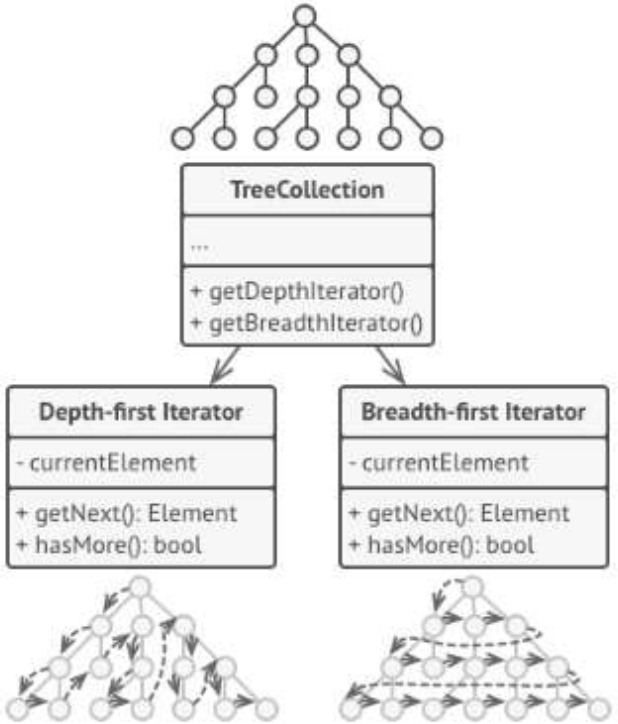
Decorator: Anexa responsabilidades adicionais a um objeto de forma dinâmica. Use para estender funcionalidades de objetos individualmente.



Various notification methods become decorators.

EXTENSÕES

Iterator: Fornece uma maneira de acessar elementos de um agregado sequencialmente sem expor sua representação subjacente. Use para percorrer coleções de objetos.



Iterators implement various traversal algorithms. Several iterator objects can traverse the same collection at

EXTENSÕES

Visitor: Representa uma operação a ser executada nos elementos de uma estrutura de objeto, permitindo que novas operações sejam definidas sem alterar as classes dos elementos. Use para adicionar novas funcionalidades a uma hierarquia de classes sem modificá-las diretamente.

OBJETIVOS DOS PADRÕES

Os padrões tem por objetivo a resolução de problemas comuns de design e promover a reutilização de código, ter flexibilidade e manutenção eficiente deste código.

Universidade Cruzeiro do Sul

